Assignment 1
Out of 50 Marks

DUE: 10 April 2026 [11:59 am]

IMPORTANT NOTES:

- This is an individual assignment.
- Homework assignments are based on assessment objectives. If an objective has been achieved, a mark will be allocated.
- **All assignments are submitted via ClickUP. See the Assignments section**.
- **You only upload your Angular App (.zip), API (.zip), and video demo (.mp4).**
- [This assignment covers L&P 02, 03, 04, 05, and 06]
- **Please execute the API migration before building your Angular application**. **See the Angular and API installation and configuration section.**
- **If you are caught plagiarising, we will give you zero per cent (0%), and you will be reported for plagiarism immediately.** We will audit historical assignments throughout the semester. **We trust that you understand the importance of this point.**

VIDEO INSTRUCTIONS:

- Make sure that everything is running when you start recording the video. The video should not be longer than **10** minutes, showing the items in the **Standard Requirements** against the **Rubric**.
- When showing something from the **Standard Requirements**, show us as much detail as required. ***See the Rubric for the assessment criteria***. For example, when assessing the "***Program Functionality,***" you must show that the CRUD functionality is working and that the data is saved and updated in the database. Similarly, for the "***Program Output,***" the correct records are displayed on the related pages per CRUD. The web pages display the right data in a valid format, demonstrating all the pages. Further, for the "***Code readability,***" we expect you to show us your code and display the organization of the code and descriptive names (*i.e., all the code used to create the program,* ***not the configuration files like package.json****, etc.*). **The same applies to the rest of the Rubric. See below.**
- If something did not work in your code, in the video, explain to us what you wanted to do and what you wanted to achieve with your approach. **This is to assess you correctly according to the Rubric**.
- **See the "Video Recording and Compression and Assignment Upload Guide" in the Assignments section on ClickUP for video recording, compression, and upload assistance.**

SUBMISSION INSTRUCTIONS:

- In this assignment, you will be given the requirements to implement.
- **Source Code:** Zip your source code files together, and for the API name it **uXXXXXXXX_HW01_API.zip**, where the XXXXXXXX is your student number, e.g., u12345678_HW01_API.zip. Further, for the Angular App, name it **uXXXXXXXX_HW01_Angular.zip**, where the XXXXXXXX is your student number, e.g., u12345678_HW01_Angular.zip.
- **Video Demo:** **Do not** zip your **video demo**. In other words, submit the actual "**.mp4**" file. Name the video demo **uXXXXXXXX_HW01.mp4**, where the XXXXXXXX is your student number, e.g., u12345678_HW01.mp4.
- If files are uploaded to the wrong upload area, we will not look for the upload. Uploads should be submitted correctly.
- **Please Note**: **If you omit the code (.zip) or the video (.mp4) submission, you will lose 50% of your assignment mark**. **If no files are uploaded (neither the .zip nor .mp4), you lose 100%. Please take this seriously and plan accordingly to submit it on time**.
- ***Note: You upload the code (.zip files) and the video demo (.mp4 file) together in the same location in the Assignment 01 Submission section. See the ClickUP information in the Assignments section (when readily available).***
- Please **do not** upload the "***node_modules"*** and "***.angular"*** folders for the **Angular App**. In other words, once you have completed your program and created your video, delete the "***node_modules"*** and "***.angular"*** folders.

As the Lecturing Team, we will reinstall the ***node_modules*** folder dependencies using the "***npm install***" terminal command, *where necessary*. **This is so you do not take long to upload your code with the video demo.**

- In addition, **do not** upload the "***bin"*** and "***obj"*** folders for the **.Net API application**. In other words, once you have completed your program and created your video, delete the "***bin"*** and "***obj"*** folders.

SUBMISSION DEADLINE: 10 April 2026

- Submission is due at **11:59 AM**
- There shall be no extensions to the deadline above.
- If homework submissions are uploaded too late, then upload errors **will** happen.
- Do not wait until the last minute to complete the assignment.
- Start working on the assignment as soon as possible.
- E-mail submissions **will not** be accepted.
- Late submissions **will not** be accepted.
- **No exceptions will be made for anyone.**

USE CASE:

Your university has noticed that students often miss important campus events, such as:

- Trends in Technology workshops
- Music festivals
- Cultural celebrations
- Competitions / Career workshops
- Sporting events

Currently, these events are promoted through posters, WhatsApp groups, and other social media channels, which often leads students to miss opportunities. The Student Council has approached you to create a proof-of-concept web application called 'CampusBuzz' (application name can be changed) to manage and promote events happening on campus.

The goal is to evaluate whether using Angular with a .NET 8 Web API (or .NET 10) improves software development productivity and provides a better user experience than traditional systems.

You have been tasked with developing:

- The backend using .NET 8 Web API (or .NET 10)
- The frontend using Angular

The application must allow administrators to manage events stored in a SQL Server database using Create, Read, Update, and Delete (CRUD) functionality.

When the application is launched, the landing page must be the "Event Listing" page, and navigation to all other pages must be done using Angular routing, subject to the restrictions described in the Standard Requirements.

STANDARD REQUIREMENTS [Figure 1]

- **Event Listing page:**
  - The event listing page should pull through the records created during the API migration using the seed data provided in the **AppDbContext.cs** file.
  - In addition, the cards that list the records should have an Edit and Delete button per event.
  - The cards should display only the Event Title, Location, and Ticket Price data from the Event table.
  - Clicking on the Delete button should delete the relevant event (e.g., Tech Workshop) and display the updated listing.
  - Clicking the Edit button should take you to the "Edit Event" page to edit the existing database record.
  - The Navigation Bar should be shared across all the created pages. The "Event Listing" link takes you to the Event Listing page (current page), and the "Add Event" link routes you to the Add Event page.

- **Edit Event page:**
  - The "Edit Event" page should allow the user to change the current event's Event Title, Location, and Ticket Price. The user can update any one, two, or all of the control values.
  - After clicking the Submit button, the record updates in the database, and the user is routed to the "Event Listing" page.

- Clicking on the Cancel button returns the user to the “Event Listing” page without making any changes to the data.

- **<u>Add Event page</u>:**
  - Once the user clicks on the “Add Event” link, they should be routed to the “Add Event” page.
  - The Submit button is disabled and will only be enabled once values are provided for all three Form Controls.
  - After clicking the Submit button, the record is created in the database, and the user is routed to the “Event Listing” page.
  - **Note =>** The newly added event should be the first item displayed in the list.
  - Clicking on the Cancel button returns the user to the “Event Listing” page without making any changes to the data.

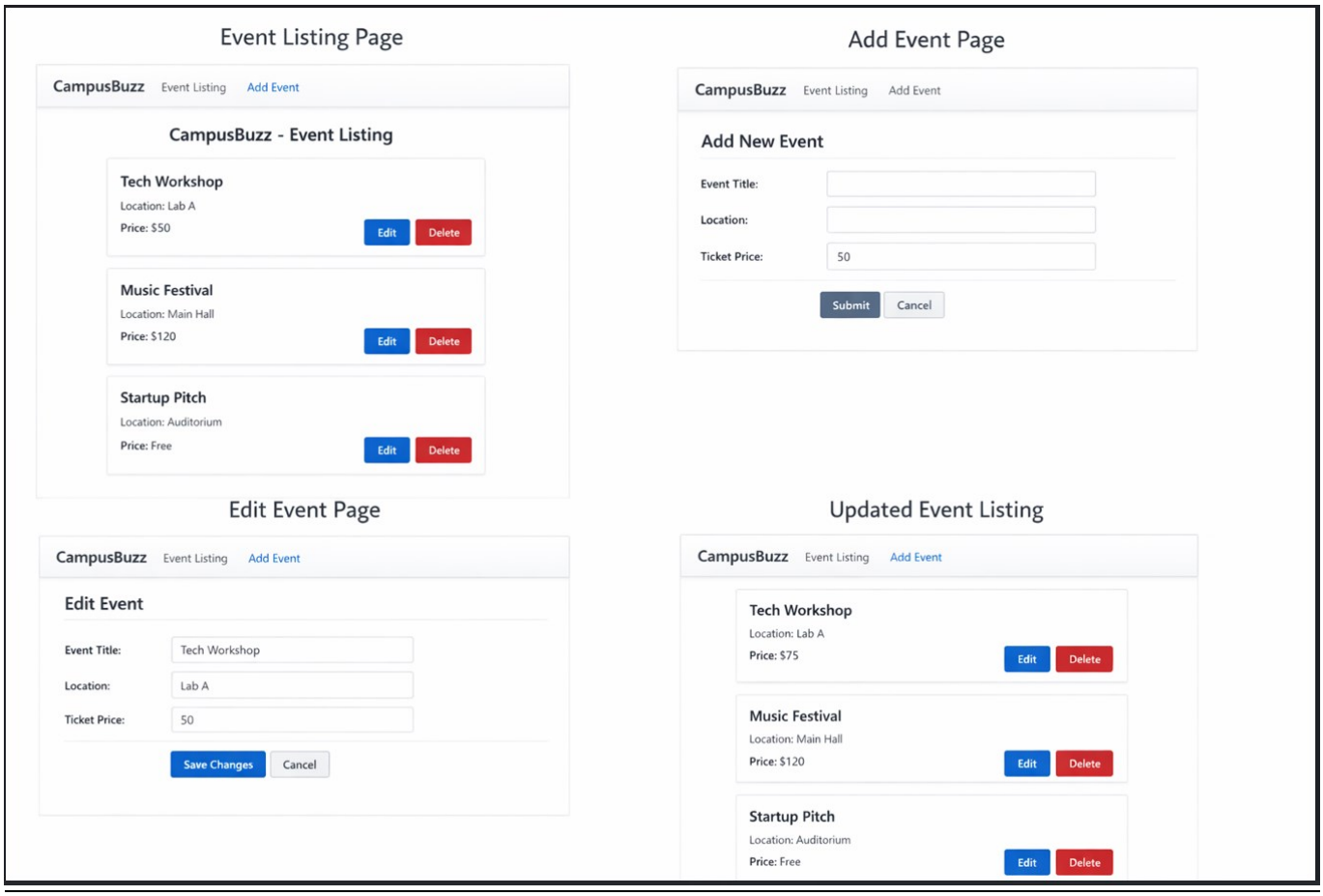

Figure 1

<u>Unit Testing:</u>

In your API backend, implement Unit Testing by completing the following xUnit test methods:

- Retrieve all Events
- Retrieve an Event by ID

Each test should verify that the returned objects are **NOT NULL** and produce a response of type **"OK"** (HTTP 200 Status Code).

Note: Ensure the proper use of Assert methods to validate the response.

ANGULAR AND API INSTALLATION AND CONFIGURATION:

- API:
  - You refer to the **Source Code** made available in the API I and API II lectures. L&P 03 and L&P 04.

- Angular:
  - You can refer to the **Source Code** made available in the Angular I and Angular II lectures. L&P 05 and L&P 06.

SUGGESTIONS:

- For the API, you will likely have 1 controller (*the API controller with endpoints (functions) to talk to the database and the Angular App*).
  - For example, an **EventController** with at least 4 endpoints (functions) to **GetAllEvents (GET)**, **AddEvent (POST)**, **EditEvent (PUT),** and **DeleteEvent (DELETE)**.
- You can design your **UI** *any way you want*, as long as it has all the controls and output required as specified in the **Standard Requirements**.
- You can develop your **API** *any way you want*, as long as it can perform the functionality required as specified in the **Standard Requirements**.

Overall Submission:

The implementation aligns with the provided rubric, as the system executes without errors, fulfills all functional requirements, including **CRUD** operations, produces correct outputs, provides a user-friendly interface, maintains clean and readable code, and is demonstrated effectively through a complete system walkthrough.

RUBRIC:

Your assignment submission will be marked according to the following rubric:

Program (50 pts)	(Exceptional)	(Very good)	(Good)	(Satisfactory)	(Poor)	(Very poor)
Program Execution	The program executes correctly with no syntax or runtime errors. *I.e. the program has no execution issues.* (10)	The program executes with one or two syntax or runtime errors. *E.g. the program loads with no crashing but displays minor bugs in the debugger.* (8)	The program executes with a few syntax or runtime errors. *E.g. A couple of runtime errors and/or the program crashes at one screen/section.* (6)	The program executes with many syntax or runtime errors. *E.g. A few runtime errors and/or the program crashes at two screens/sections.* (5)	The program executes with major errors. *E.g. The program can execute, however, it is plagued with the runtime or syntax errors, or the program keeps crashing during use.* (3)	The program does not execute. *I.e. The application fails to run.* (0)
Program Functionality	Program functionality is in line with the requirements. *I.e. the program has all the correct functionality implemented.* (10)	Program functionality has one minor inconsistency. *E.g. One of the functional requirements is incorrect.* (8)	Program functionality has a few minor inconsistencies. *E.g. Two of the functional requirements are incorrect or one is missing.* (6)	Program functionality has many inconsistencies. *E.g. Some of the functional requirements are incorrect or half is missing.* (5)	Program functionality has major inconsistencies. *E.g. Most of the functional requirements are incorrect or missing.* (3)	Program functionality is missing. *E.g. None of the functionality works or all the functionality is missing.* (0)
Program Output	The program displays the correct output in line with the requirements. *I.e. It produces the same output as required.* (10)	The program has one or two very minor output discrepancies. *I.e. It produces output with barely noticeable inconsistencies. E.g. one or two formatting issues.* (8)	The program has a few output discrepancies. *I.e. It produces output with easily noticeable inconsistencies. E.g. The program does not return some of the data or there are a few formatting issues.* (6)	The program has many output discrepancies. *I.e. It produces output with many noticeable inconsistencies. E.g. The program does not return half of the data or there are plenty of formatting issues.* (5)	The program has major output discrepancies. *I.e. The output is plagued with inconsistencies. E.g. The program does not return most of the data or there are substantial formatting issues.* (3)	Output is incorrect. *E.g. The program does not provide any of the requested output or all the formatting is not as requested in the requirements.* (0)
Program Interface (UI)	The program interface is professionally done. I.e. *The interface is implemented correctly and looks very good.* (5)	The program interface is done well. *I.e. The interface is implemented correctly and looks good. E.g. One or two styling/layout issues.* (4)	N/A	The program interface is good enough. *I.e. The interface is implemented correctly and looks okay. E.g. A few styling/layout issues.* (3)	The program interface is poorly done. *I.e. The interface is mostly incorrect or looks poorly done. E.g. The layout is mostly incorrect or has plenty of styling issues.* (2)	The program interface is very poor. *I.e. The interface is entirely incorrect or is very poorly done. E.g. The layout is completely incorrect or the styling is missing.* (0)
Code Readability	The program code is well organized and makes good use of white space. Variables have	The program code is organized and makes use of white space. Variables have descriptive names.	N/A	Program code is mostly organized and makes use of white space. Most variables have descriptive	Program code is somewhat organized, and not easy to read and understand. *E.g. There are plenty of variable naming convention issues*	Program code is difficult to read. *E.g. Variable naming conventions are*

Program (50 pts)	(Exceptional)	(Very good)	(Good)	(Satisfactory)	(Poor)	(Very poor)
	descriptive names. I.e. *There is nothing to fault on.* (5)	*E.g. There are one or two variable naming convention issues or white space issues.* (4)		names. *E.g. There are a few variable naming convention issues or program code organization that could be improved.* (3)	*or the code is challenging to follow.* (2)	*missing or the code is hard to follow.* (0)
Video Demonstration	The program is exceptionally well presented. I.e. *The student demonstrated and displayed all the required functionality, output, interfaces, and code.* (10)	The program is well presented. *E.g. The student demonstrated and displayed all the required functionality, output, interfaces, and code. However, one of the descriptions or illustrations was lacking.* (8)	The program presentation is good. *E.g. The student demonstrated and displayed most of the required functionality, output, interfaces, and code. However, two of the functionality, output, interfaces and code descriptions or illustrations were lacking or missing.* (6)	The program presentation is adequate. *E.g. The student demonstrated and displayed most of the required functionality, output, interfaces, and code. However, a few to half of the functionality, output, interfaces and code descriptions or illustrations were lacking/missing.* (5)	The program is presented poorly. *E.g. The student demonstrated and displayed a few of the required functionality, output, interfaces, and code. However, most functionality, output, interfaces, and code descriptions or illustrations were lacking/missing.* (3)	The program has barely been presented or has been presented very poorly. *E.g. The student failed to demonstrate and display the required functionality, output, interfaces, and code or it was missing.* (0)